

REPORTE DE PRÁCTICA

Control de acceso con Arduino UNO y lector RFID RC522 mediante bus SPI

Comunicación SPI, lectura de UID y manejo de tiempos con millis()

Alumno(s): _____IVAN LUNA Y DANIEL BUSTAMANTE

Fecha: _____28/09/2026_____

1. Objetivo

Comunicar un Arduino UNO con un lector RFID RC522 por medio del bus SPI, leer el número único (UID) de una tarjeta, mostrarlo en el Monitor Serie y usarlo para simular un sistema de control de acceso con indicadores luminosos (LED verde y rojo).

2. Materiales

- 1 placa Arduino UNO con cable USB
- 1 módulo lector RFID RC522
- 1 tarjeta y 1 llavero RFID (MIFARE 13.56 MHz)
- 1 LED verde y 1 LED rojo
- 2 resistencias de 220 Ω
- 1 protoboard y cables de conexión
- Arduino IDE con la librería MFRC522 (instalada desde el Library Manager)

Advertencia: el módulo RC522 se alimenta con 3.3 V. Conectarlo a 5 V puede dañarlo.

3. Marco teórico

3.1 Bus SPI

SPI (Serial Peripheral Interface) es un bus serie síncrono de cuatro líneas: SCK (reloj), MOSI (datos del maestro al esclavo), MISO (datos del esclavo al maestro) y SS (selección del esclavo). El Arduino actúa como maestro y el RC522 como esclavo. Si la línea MISO se desconecta, el maestro no recibe respuesta y lee valores como 0x00 o 0xFF.

3.2 RFID y UID

El RC522 genera un campo electromagnético de 13.56 MHz que energiza la tarjeta y le permite responder. Cada tarjeta tiene un identificador único (UID) de 4 bytes, que se usa aquí para decidir si se concede o se niega el acceso.

3.3 Temporización con millis()

La función delay() detiene el programa mientras espera. En cambio, millis() devuelve el tiempo transcurrido desde el arranque; al comparar la hora actual con la hora en que se encendió el LED, el programa puede apagarlo cuando pasen los 3000 ms y seguir leyendo tarjetas mientras tanto.

4. Conexiones

4.1 Módulo RC522 a Arduino UNO

Pin RC522	Pin Arduino UNO	Función
SDA (SS)	D10	Selección del esclavo

Pin RC522	Pin Arduino UNO	Función
SCK	D13	Reloj SPI
MOSI	D11	Datos Arduino → RC522
MISO	D12	Datos RC522 → Arduino
IRQ	Sin conectar	No se utiliza
GND	GND	Tierra común
RST	D9	Reinicio del lector
3.3V	3.3V	Alimentación (nunca 5 V)

4.2 LED indicadores

LED	Pin Arduino	Resistencia	Cátodo a
Verde (acceso permitido)	D6	220 Ω en serie	GND
Rojo (acceso denegado)	D7	220 Ω en serie	GND

La pata larga (ánodo) de cada LED va hacia la resistencia y el pin del Arduino; la pata corta (cátodo) va al riel de GND de la protoboard.

4.3 Diagrama de conexiones

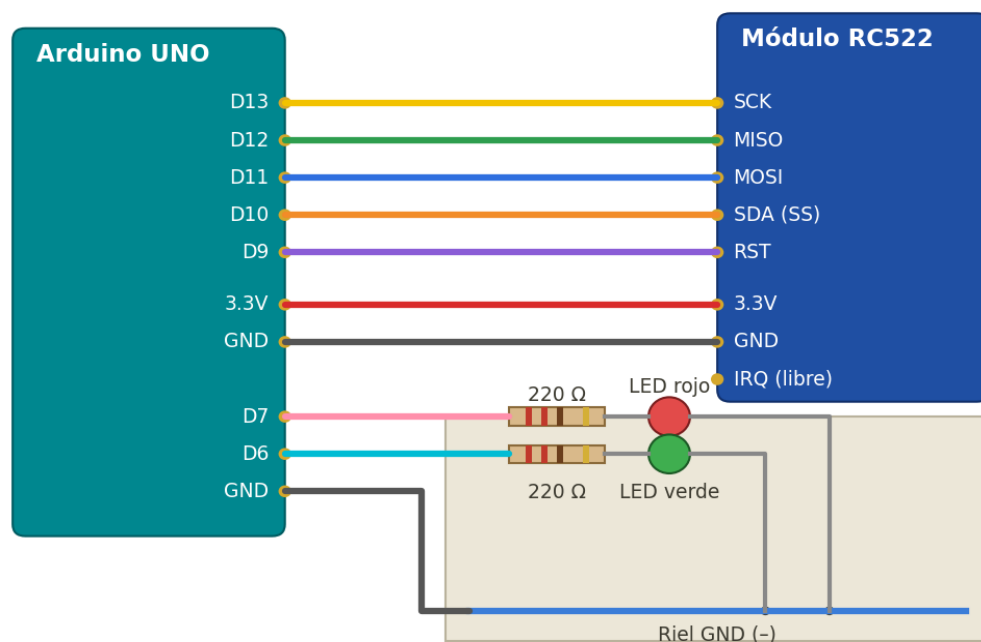


Figura 1. Diagrama de conexiones del circuito.

5. Funcionamiento del programa

1. Al iniciar, se configuran los pines de los LED, se abre el Monitor Serie a 9600 baudios y se inicializan el bus SPI y el lector.
2. Se lee el registro de versión del chip (VersionReg). Si el valor es 0x00 o 0xFF no hay comunicación y se muestra un mensaje de error; en caso contrario se confirma la comunicación.
3. En el ciclo principal se revisa si hay una tarjeta nueva y, si la hay, se lee su UID y se imprime con cada byte separado por un espacio (ejemplo: UID: C7 B4 27 3C).
4. El UID leído se compara byte por byte con el UID autorizado guardado en el programa.
5. Si coincide, se muestra ACCESO PERMITIDO y se enciende el LED verde por 3 segundos. Si no coincide, se muestra ACCESO DENEGADO y se enciende el LED rojo por 3 segundos.
6. El apagado de los LED se controla con millis(), por lo que el programa nunca se bloquea y sigue leyendo tarjetas mientras el LED está encendido.

6. Código fuente

UID autorizado: C7 B4 27 3C (tarjeta elegida como autorizada).

```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN    10
#define RST_PIN   9
#define LED_VERDE 6
#define LED_ROJO  7

const unsigned long TIEMPO_LED = 3000;    // 3 segundos

byte uidAutorizado[] = {0xC7, 0xB4, 0x27, 0x3C};
const byte TAM_UID = sizeof(uidAutorizado);

MFRC522 rfid(SS_PIN, RST_PIN);

unsigned long inicioLed = 0;
bool ledActivo = false;
int pinActivo = -1;

void encenderLed(int pin) {
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, LOW);
    digitalWrite(pin, HIGH);
    pinActivo = pin;
    inicioLed = millis();
    ledActivo = true;
}

void actualizarLed() {
    if (ledActivo && (millis() - inicioLed >= TIEMPO_LED)) {
        digitalWrite(pinActivo, LOW);
        ledActivo = false;
        pinActivo = -1;
    }
}
```

```

void imprimirUID(byte *uid, byte tam) {
    Serial.print("UID:");
    for (byte i = 0; i < tam; i++) {
        Serial.print(' ');
        if (uid[i] < 0x10) Serial.print('0');
        Serial.print(uid[i], HEX);
    }
    Serial.println();
}

bool uidCoincide(byte *uid, byte tam) {
    if (tam != TAM_UID) return false;
    for (byte i = 0; i < tam; i++) {
        if (uid[i] != uidAutorizado[i]) return false;
    }
    return true;
}

bool lectorResponde() {
    byte version = rfid.PCD_ReadRegister(MFRC522::VersionReg);
    Serial.print("Version del chip: 0x");
    Serial.println(version, HEX);
    return (version != 0x00 && version != 0xFF);
}

void setup() {
    Serial.begin(9600);
    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_ROJO, OUTPUT);
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, LOW);

    SPI.begin();
    rfid.PCD_Init();
    delay(50);

    if (lectorResponde()) {
        Serial.println("Lector RC522 detectado: comunicacion SPI OK");
        Serial.println("Acerque una tarjeta...");
    } else {
        Serial.println("ERROR: no hay comunicacion con el RC522.");
        Serial.println("Revise cableado (MISO, MOSI, SCK, SDA, RST, 3.3V, GND).");
    }
}

void loop() {
    actualizarLed();

    if (!rfid.PICC_IsNewCardPresent()) return;
    if (!rfid.PICC_ReadCardSerial()) return;

    imprimirUID(rfid.uid.uidByte, rfid.uid.size);

    if (uidCoincide(rfid.uid.uidByte, rfid.uid.size)) {
        Serial.println("ACCESO PERMITIDO");
        encenderLed(LED_VERDE);
    } else {
        Serial.println("ACCESO DENEGADO");
        encenderLed(LED_ROJO);
    }

    rfid.PICC_HaltA();
}

```

```
rfid.PCD_StopCrypto1();  
}
```

7. Pruebas y resultados

7.1 UID de las tarjetas

Elemento	UID	Estado
Tarjeta	C7 B4 27 3C	Autorizada
Llavero	— — — —	No autorizado

7.2 Pruebas realizadas

N.º	Prueba	Resultado esperado	Resultado observado
1	Acercar la tarjeta autorizada	Monitor: ACCESO PERMITIDO. LED verde enciende y se apaga solo a los 3 s.	
2	Acercar la tarjeta no autorizada	Monitor: ACCESO DENEGADO. LED rojo enciende por 3 s y se apaga solo.	
3	Con el LED encendido, acercar la tarjeta no autorizada	ACCESO DENEGADO aparece de inmediato; el sistema no se bloquea (millis()).	
4	Desconectar MISO (D12) y reiniciar el Arduino	Versión del chip 0x00 o 0xFF y mensaje de error de comunicación. Al reconectar y reiniciar, vuelve a funcionar.	

Nota: complete la columna de resultado observado y el UID del llavero con lo obtenido en su práctica.

8. Conclusiones

El bus SPI permite comunicar el Arduino con el módulo RC522 usando cuatro líneas (SS, SCK, MOSI y MISO) más alimentación y tierra. La lectura del registro de versión es una forma sencilla de verificar que la comunicación funciona, y su fallo al desconectar MISO demuestra la importancia de cada línea del bus.

El UID de cada tarjeta permite identificar usuarios y simular un control de acceso comparando el valor leído con uno autorizado. Finalmente, el uso de millis() en lugar de delay() hace posible mantener un LED encendido durante un tiempo determinado sin detener la lectura de nuevas tarjetas, lo que es fundamental en sistemas que deben atender varias tareas a la vez.